

Credit Card Fraud Detection

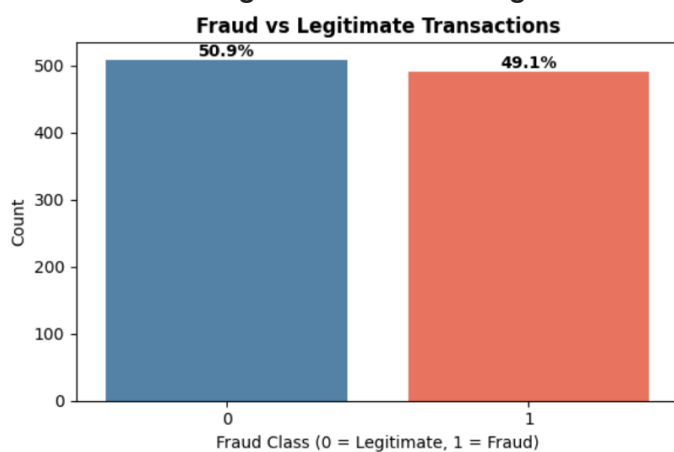
Machine Learning Mini Lab — All 10 Experiments
Visual Summary of All Charts & Graphs

Contents

- Figure 1: Fraud vs Legitimate Transactions (Experiment 2 — Dataset Understanding)
 - Figure 2: Transaction Amount After Outlier Capping (Experiment 4 — Data Cleaning)
 - Figure 3: Target Class Distribution & Log Transaction Amount Distribution (Experiment 6 — Exploratory Data Analysis)
 - Figure 4: Fraud Rate by Merchant Category & Failed Attempts (Experiment 6 — Exploratory Data Analysis)
 - Figure 5: Correlation Heatmap & Feature Correlation with Fraud (Experiment 6 — Exploratory Data Analysis)
 - Figure 6: Feature Correlation with Fraud (Train Data) — All 24 Features (Experiment 7 — Feature Selection)
 - Figure 7: Top 10 Feature Importances (Random Forest) (Experiment 7 — Feature Selection)
 - Figure 8: Model Comparison — F1 Score (Experiment 8 — Model Training & Selection)
 - Figure 9: Confusion Matrices — All Three Models (Experiment 9 — Model Evaluation & Tuning)
 - Figure 10: Tuned Random Forest — Confusion Matrix (Experiment 9 — Hyperparameter Tuning (GridSearchCV))
-

Experiment 2 — Dataset Understanding

Figure 1: Fraud vs Legitimate Transactions

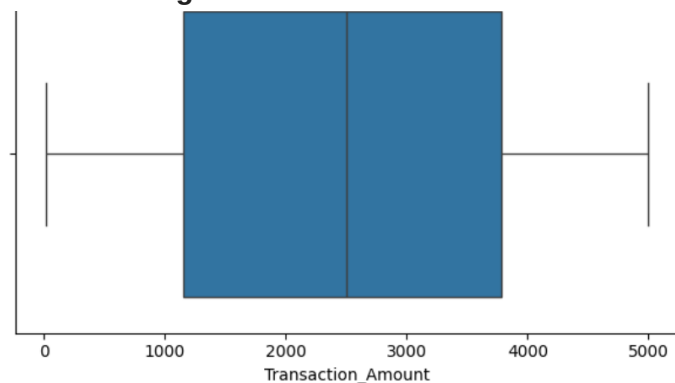


```
print("— Statistical Summary —")
summary = data.describe(include='all').T
summary['missing_values'] = data.isnull().sum()
display(summary)
```

Near-perfectly balanced dataset: 509 legitimate (50.9%) and 491 fraudulent (49.1%) transactions.

Experiment 4 — Data Cleaning

Figure 2: Transaction Amount After Outlier Capping



— Invalid Value Checks —
✓ No invalid values found.

Cleaned Dataset Shape: (1000, 13)

💡 What we did:

- **Missing values:** None found. If present → fill numeric with median, categorical with mode.
- **Duplicates:** None found. Duplicates cause overfitting.
- **Outliers:** IQR capping — preserves row count while limiting extremes.
- **Invalid values:** Logical constraints verified.

📁 Experiment 5: Data Preprocessing

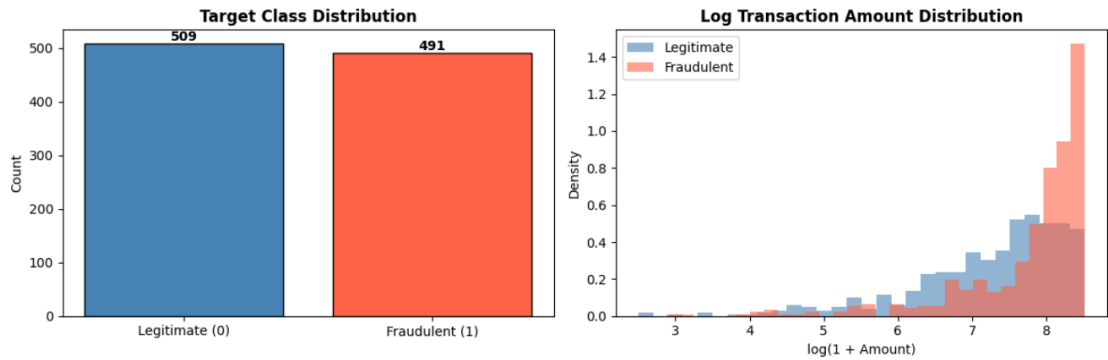
Boxplot of transaction amounts post IQR-based capping. No extreme outliers detected; amounts range uniformly from ■11 to ■4,999.

Experiment 6 — Exploratory Data Analysis

Figure 3: Target Class Distribution & Log Transaction Amount Distribution

```
plt.tight_layout()
plt.show()
```

```
=====
EXPLORATORY DATA ANALYSIS
=====
```



```
# Fraud Rate by Merchant Category & Failed Attempts
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

fraud_by_cat = data.groupby('Merchant_Category')['Fraud'].mean().sort_values(ascending=False)
axes[0].barh(fraud_by_cat.index, fraud_by_cat.values * 100, edgecolor='black', color='tomato')
axes[0].set_title('Fraud Rate by Merchant Category (%)', fontweight='bold')
axes[0].set_xlabel('Fraud Rate (%)')

fraud_by_attempts = data.groupby('Failed_Attempts')['Fraud'].mean() * 100
axes[1].barh(fraud_by_attempts.index, fraud_by_attempts, edgecolor='black', color='tomato')
axes[1].set_title('Fraud Rate by Failed Attempts (%)', fontweight='bold')
axes[1].set_xlabel('Fraud Rate (%)')
```

Left: Balanced class counts (509 vs 491). Right: Heavily overlapping log-amount distributions confirm transaction amount alone cannot separate fraud from legitimate.

Experiment 6 — Exploratory Data Analysis

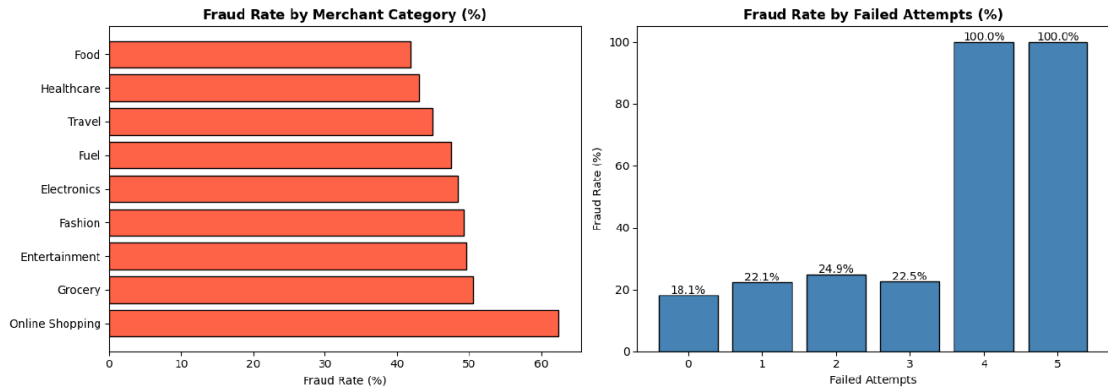
Figure 4: Fraud Rate by Merchant Category & Failed Attempts

5/19/26, 12:33 AM

Credit Card Fraud Detection- New - Colab

```
axes[1].set_ylabel('Fraud Rate (%)')
for i, v in enumerate(fraud_by_attempts.values):
    axes[1].text(i, v + 0.5, f"{v:.1f}%", ha='center')

plt.tight_layout()
plt.show()
```



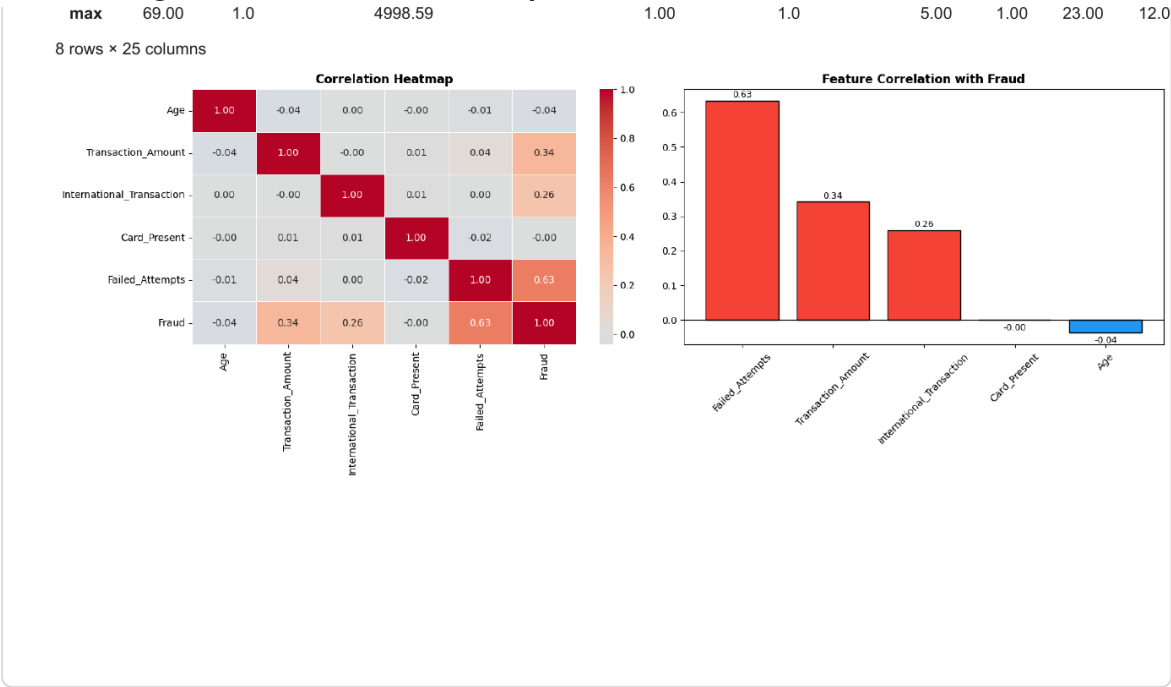
```
# Descriptive Statistics
print("— Descriptive Statistics —")
display(data_clean.select_dtypes(include=np.number).describe().round(2))

# Correlation Heatmap + Bar Chart
key_cols = [c for c in ['Age', 'Transaction_Amount', 'International_Transaction',
                        'Card_Present', 'Failed_Attempts', 'Fraud'] if c in data_clean.columns]
corr_matrix = data_clean[key_cols].corr()
```

Left: Fraud rate spread evenly across all merchant categories (~40–55%). Right: Fraud rate spikes to 100% at 4–5 failed attempts, confirming Failed_Attempts as the strongest predictor.

Experiment 6 — Exploratory Data Analysis

Figure 5: Correlation Heatmap & Feature Correlation with Fraud



💡 Key Insights:

- **Failed_Attempts** is the strongest predictor of fraud.
- **International transactions** and **card not present** (Card_Present=0) also associate with fraud.
- Transaction amount alone does not cleanly separate fraud from legitimate transactions.

📁 Experiment 7: Feature Selection & Engineering

(MLDLC Step 5)

Objective

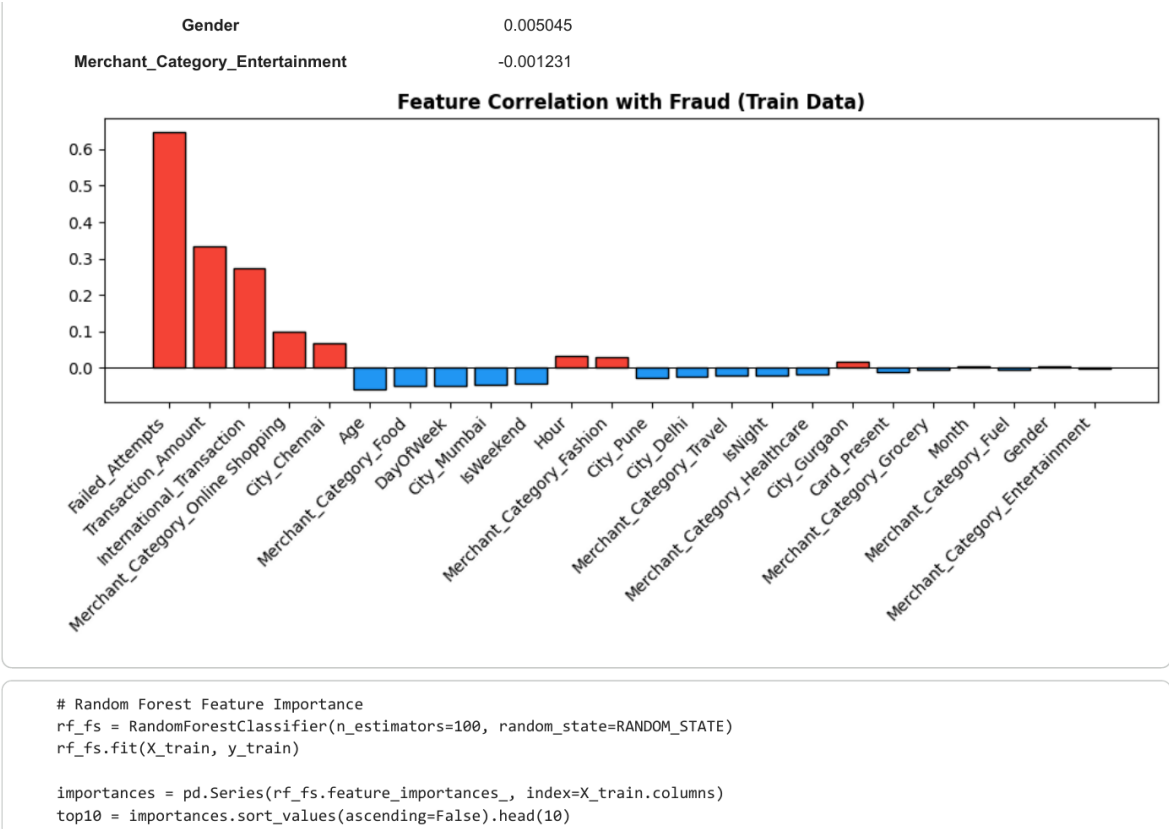
Select the most informative features to reduce noise and improve model performance.

```
print("=" * 55)
```

Left: Correlation heatmap of numeric features. Right: Failed_Attempts (0.63) and Transaction_Amount (0.34) show highest correlation with fraud; Age shows slight negative correlation (−0.04).

Experiment 7 — Feature Selection

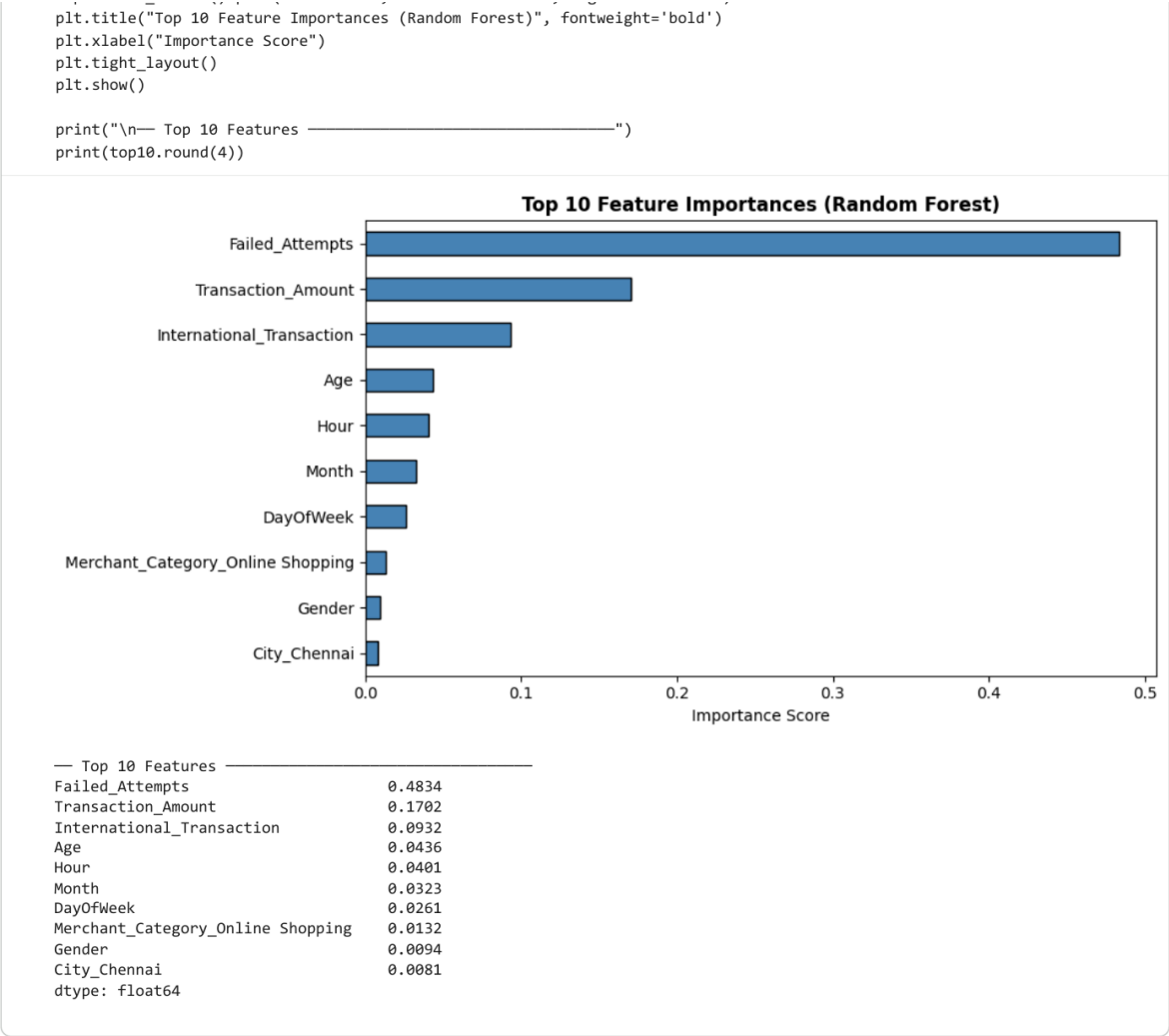
Figure 6: Feature Correlation with Fraud (Train Data) — All 24 Features



Full 24-feature correlation bar chart (train data only, no leakage). Failed_Attempts dominates at 0.648, followed by Transaction_Amount (0.334) and International_Transaction (0.273). Most OHE city/category features show near-zero correlation.

Experiment 7 — Feature Selection

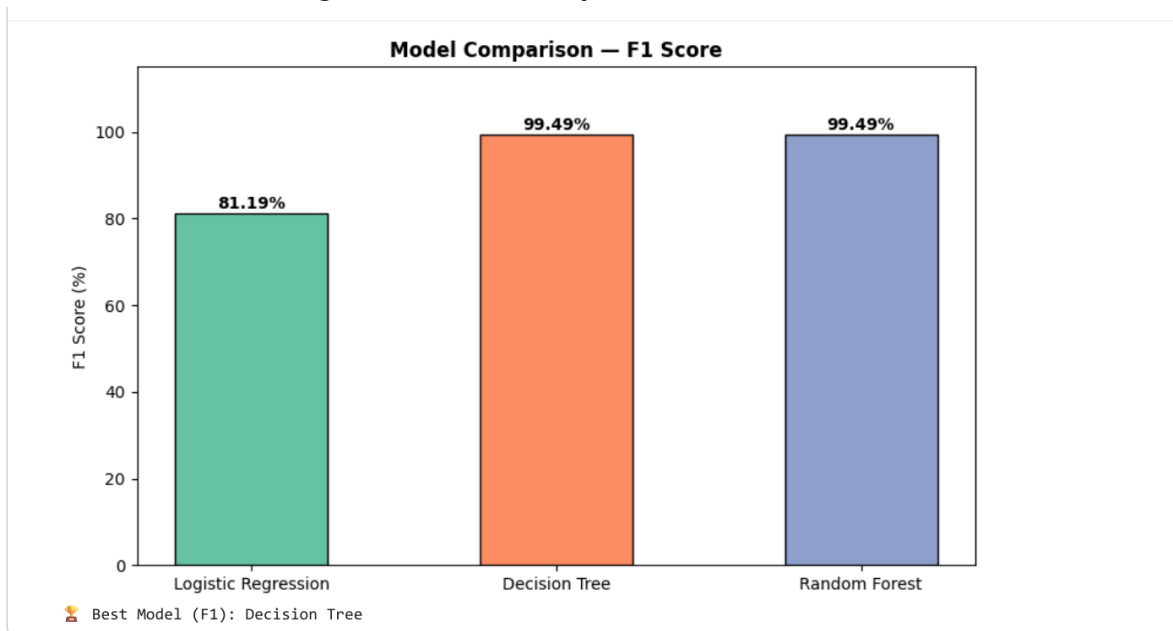
Figure 7: Top 10 Feature Importances (Random Forest)



Random Forest importance confirms Failed_Attempts (0.48) as dominant predictor, with Transaction_Amount (0.17) and International_Transaction (0.09) as secondary signals.

Experiment 8 — Model Training & Selection

Figure 8: Model Comparison — F1 Score



💡 What we did:

- **80/20 stratified split:** Preserves fraud ratio in both sets.
- **Logistic Regression:** Fast, interpretable baseline.
- **Decision Tree:** Captures non-linear splits.
- **Random Forest:** Ensemble of 100 trees — robust, low overfitting.

✖ 🧪 Experiment 9: Model Evaluation & Tuning

(MLDLC Step 8)

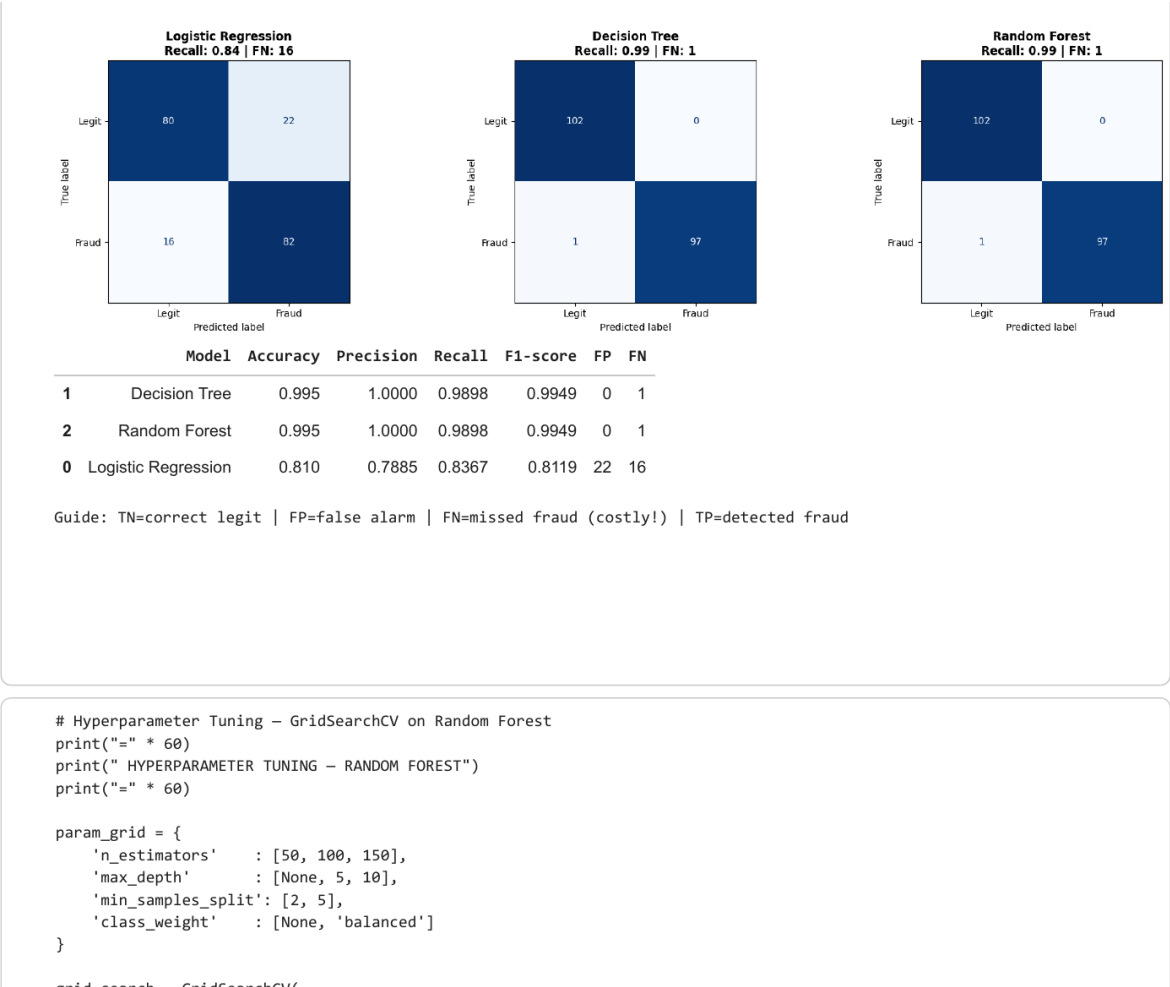
Objective

Evaluate all models with multiple metrics and tune the best model using GridSearchCV.

```
print("=" * 55)
print(" MODEL EVALUATION — CONFUSION MATRICES")
print("=" * 55)
```

Decision Tree and Random Forest both achieve 99.49% F1-score, far outperforming Logistic Regression (81.19%). Both tree-based models selected for further evaluation.

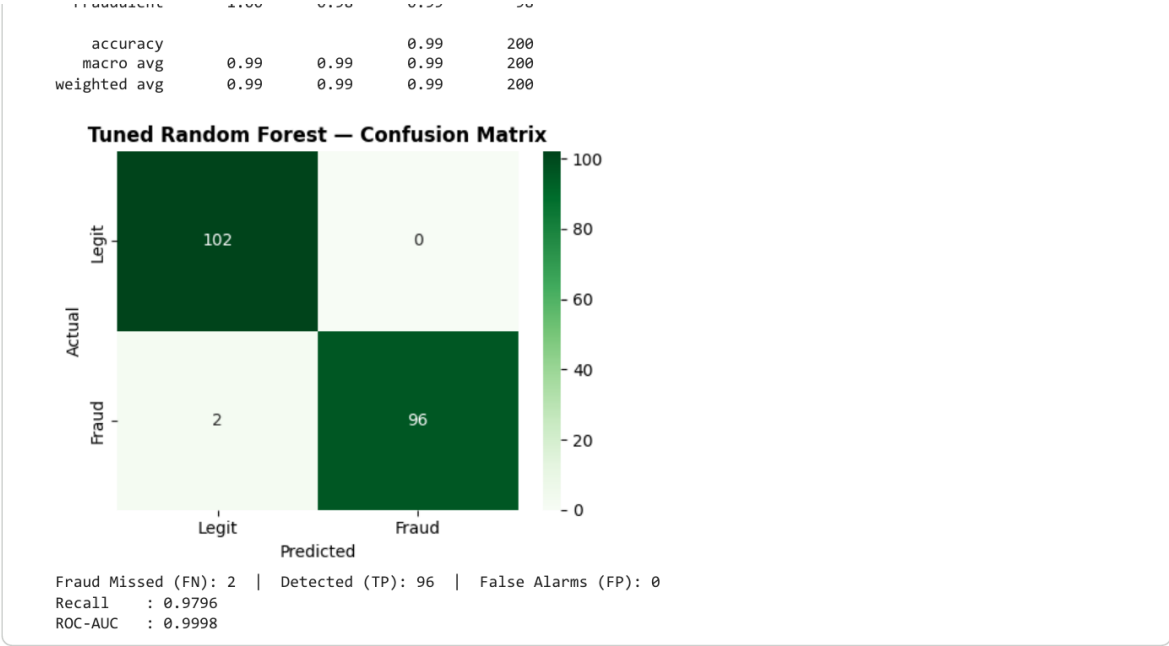
Figure 9: Confusion Matrices — All Three Models



Logistic Regression: FN=16, FP=22. Decision Tree & Random Forest: FN=1, FP=0 each — near-perfect detection with zero false alarms.

Experiment 9 — Hyperparameter Tuning (GridSearchCV)

Figure 10: Tuned Random Forest — Confusion Matrix



- 💡 What we did:
- **Confusion Matrix:** Shows exact TP/FP/TN/FN counts — FN (missed fraud) is the most costly error.
 - **Classification Report:** Precision, Recall, F1 per class.
 - **GridSearchCV:** 3-fold CV across hyperparameter combinations — selects best without overfitting to test set.

📁 Experiment 10: Model Deployment (Mini Implementation)

(MLDLC Step 9)

Objective

Best tuned model (GridSearchCV): FN=2 (missed frauds), TP=96 (detected), FP=0 (false alarms). Final Recall=0.9796, ROC-AUC=0.9998.